HOSPITAL MANAGEMENT SYSTEM

C++ SOURCE CODE

```
001  #include <iostream>
002  #include <vector>
003  #include <memory>
004  #include <string>
005  #include <limits>
006  using namespace std;
007
008  // Abstract base class: abstraction and encapsulation
009  class Person {
010  protected:
011      int id;
012      string name;
013  public:
014      Person(int i, string n) : id(i), name(n) {}
015      virtual void display() const = 0; // Pure virtual function
016      virtual ~Person() = default;
017      int getId() const { return id; }
018  };
019
020  // Inheritance: Patient is a Person
021  class Patient : public Person {
022      int age;
023      string disease;
024  public:
025      Patient(int i, string n, int a, string d)
026          : Person(i, n), age(a), disease(d) {}
027      void display() const override {
028          cout << "Patient ID: " << id << " | Name: " << name
029               << " | Age: " << age << " | Disease: " << disease << '\n';
030      }
031  };
032
033  // Inheritance: Doctor is a Person
034  class Doctor : public Person {
035      string specialization;
036  public:
037      Doctor(int i, string n, string s)
038          : Person(i, n), specialization(s) {}
039      void display() const override {
040          cout << "Doctor ID: " << id << " | Name: " << name
041               << " | Specialization: " << specialization << '\n';
042      }
043  };
044
045  class Hospital {
046      vector<unique_ptr<Patient>> patients;
047      vector<unique_ptr<Doctor>> doctors;
048  public:
049      void addPatient(int id, const string& name, int age, const string& disease) {
```

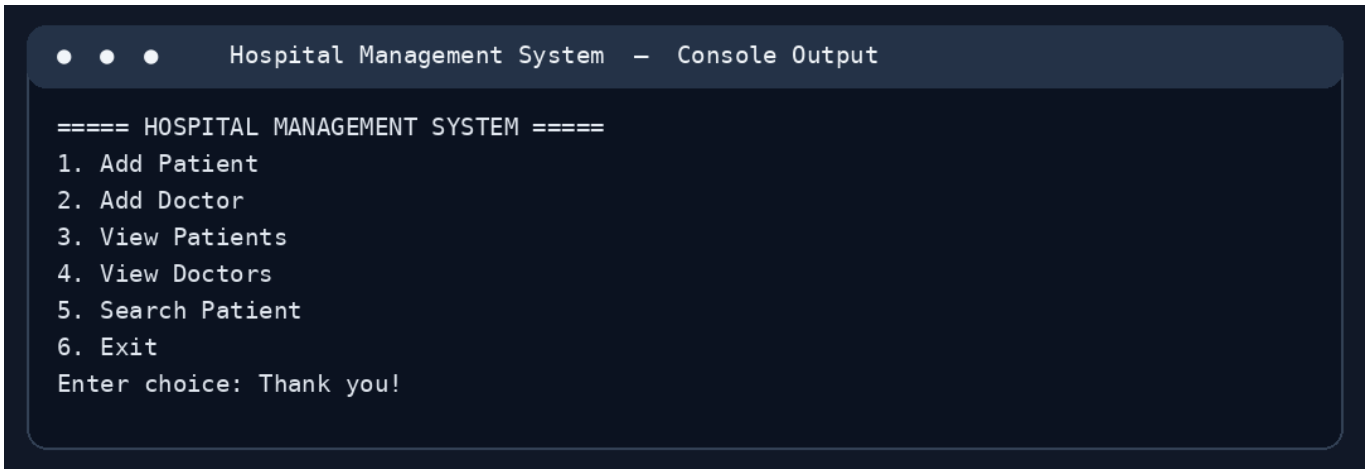
```
        for (const auto& p : patients) {
            if (p->getId() == id) { cout << "Patient ID already exists.\n"; return; }
        }
        patients.push_back(make_unique<Patient>(id, name, age, disease));
        cout << "Patient added successfully.\n";
    }
    void addDoctor(int id, const string& name, const string& specialty) {
        for (const auto& d : doctors) {
            if (d->getId() == id) { cout << "Doctor ID already exists.\n"; return; }
        }
        doctors.push_back(make_unique<Doctor>(id, name, specialty));
        cout << "Doctor added successfully.\n";
    }
    void showPatients() const {
        if (patients.empty()) { cout << "No patients found.\n"; return; }
        for (const auto& p : patients) {
            const Person& person = *p; // Runtime polymorphism
            person.display();
        }
    }
    void showDoctors() const {
        if (doctors.empty()) { cout << "No doctors found.\n"; return; }
        for (const auto& d : doctors) {
            const Person& person = *d; // Runtime polymorphism
            person.display();
        }
    }
    void searchPatient(int id) const {
        for (const auto& p : patients) {
            if (p->getId() == id) { p->display(); return; }
        }
        cout << "Patient not found.\n";
    }
};

int readInt(const string& prompt) {
    int value;
    while (true) {
        cout << prompt;
        if (cin >> value) {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
        cout << "Please enter a valid number.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}
string readText(const string& prompt) {
```

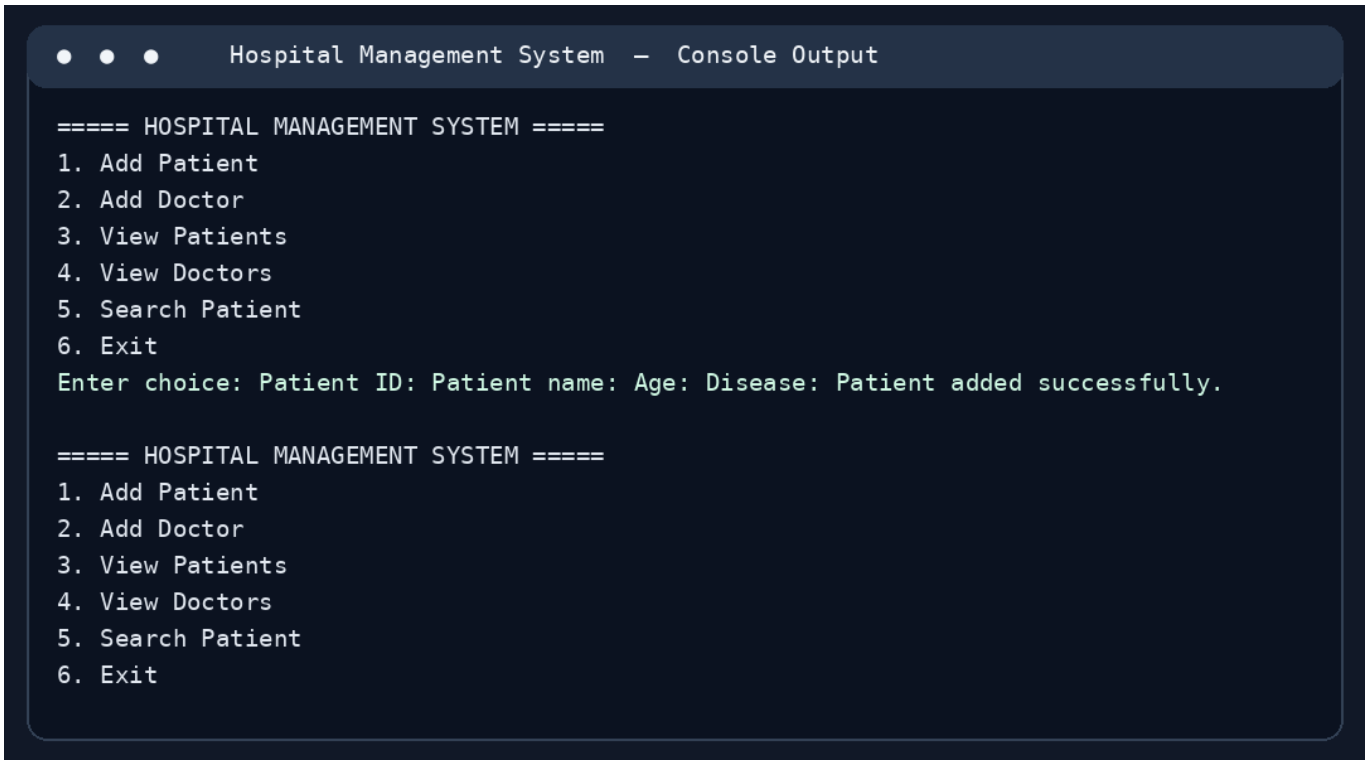
```
    string value;
    do { cout << prompt; getline(cin, value); }
    while (value.empty());
    return value;
}
int main() {
    Hospital hospital;
    while (true) {
        cout << "\n===== HOSPITAL MANAGEMENT SYSTEM =====\n"
             << "1. Add Patient\n2. Add Doctor\n3. View Patients\n"
             << "4. View Doctors\n5. Search Patient\n6. Exit\n";
        int choice = readInt("Enter choice: ");
        if (choice == 1) {
            int id = readInt("Patient ID: ");
            string name = readText("Patient name: ");
            int age = readInt("Age: ");
            if (age < 0 || age > 130) { cout << "Invalid age.\n"; continue; }
            string disease = readText("Disease: ");
            hospital.addPatient(id, name, age, disease);
        } else if (choice == 2) {
            int id = readInt("Doctor ID: ");
            string name = readText("Doctor name: ");
            string specialty = readText("Specialization: ");
            hospital.addDoctor(id, name, specialty);
        } else if (choice == 3) hospital.showPatients();
        else if (choice == 4) hospital.showDoctors();
        else if (choice == 5) hospital.searchPatient(readInt("Patient ID to search: "));
        else if (choice == 6) { cout << "Thank you!\n"; break; }
        else cout << "Invalid choice. Try again.\n";
    }
}
```

PROGRAM OUTPUT SCREENSHOTS

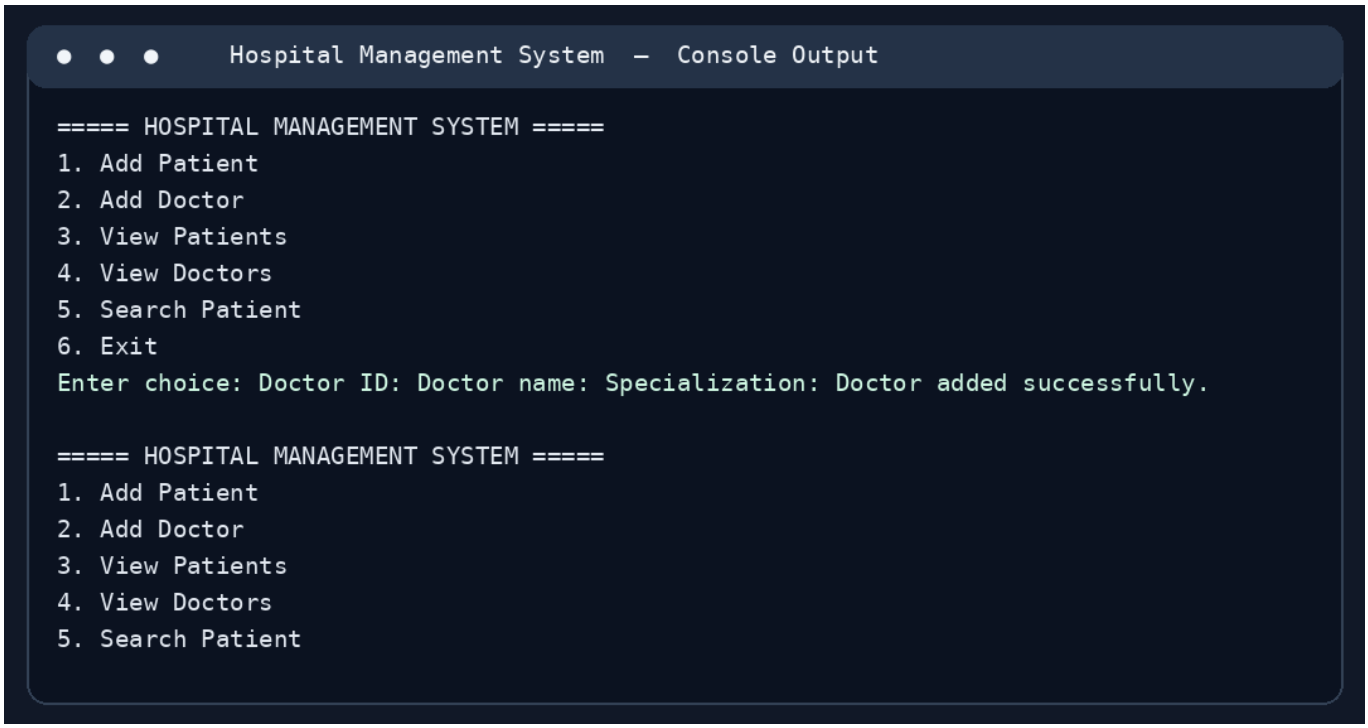
Output 1: Main Menu



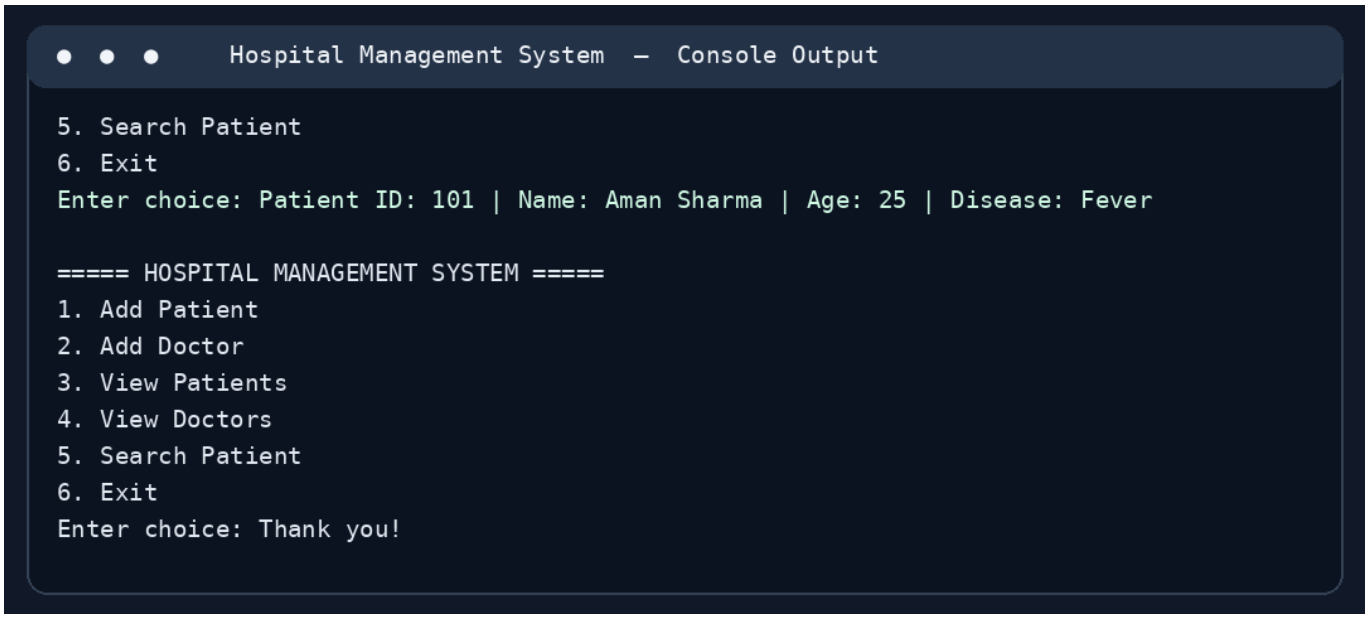
Output 2: Add Patient



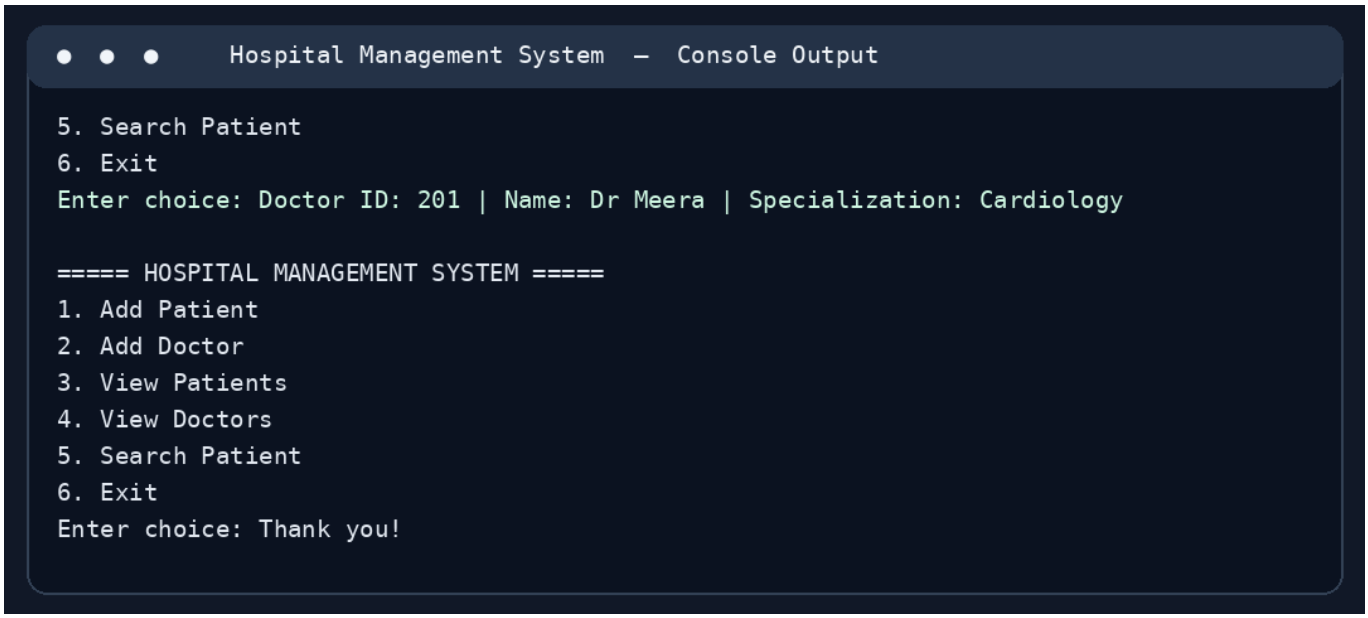
Output 3: Add Doctor



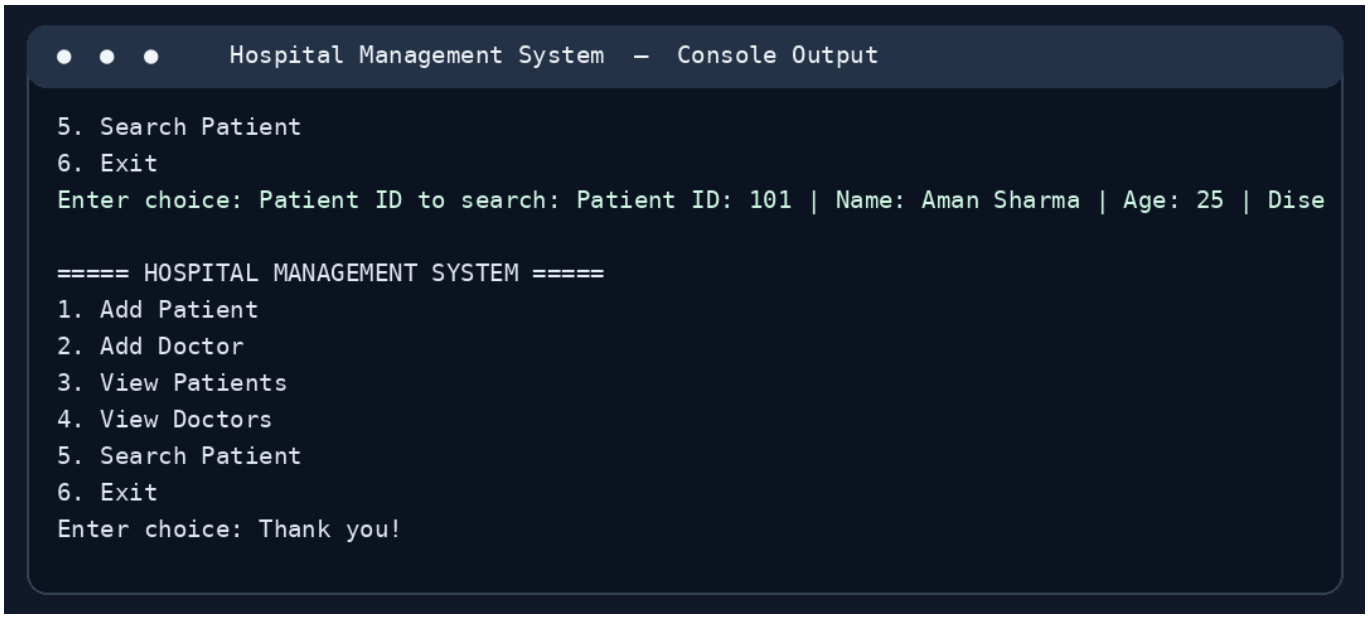
Output 4: View Patients



Output 5: View Doctors



Output 6: Search Patient



Output 7: Patient Not Found

